PROJECT REPORT FOR

FOUNDATIONS OF ARTIFICIAL INTELLIGENCE

INTELLITRANSPLANT

AI-ASSISTED ORGAN TRANSPLANT COORDINATION AND TRANSPORT PLANNING

A PROJECT REPORT

Submitted by

HARSHINI K P– 240701183

HARSHINI U – 240701185

Department of Computer Science and Engineering

RAJALAKSHMI ENGINEERING COLLEGE

October 2026



BONAFIDE CERTIFICATE

Certified that this project report entitled "INTELLITRANSPLANT: AI-ASSISTED ORGAN TRANSPLANT COORDINATION AND TRANSPORT PLANNING" is the Bonafide work of "HARSHINI KP -240701183 & HARSHINI U 240701185" who carried out the project work under my supervision.

B.DEEPA

ASSISTANT PROFESSOR SG

Dept. of Computer Science and Engineering,

Rajalakshmi Engineering College

Chennai

SIGNATURE OF THE FACULTY INCHARGE

Submitted for the Practical Examination held on _____

SIGNATURE OF THE INTERNAL EXAMINER

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	4
1	INTRODUCTION	5
2	LITERATURE REVIEW	6
3	PROPOSED SYSTEM	7
4	MODULES DESCRIPTION	9
5	IMPLEMENTATION AND RESULTS	15
6	CONCLUSION AND FUTURE WORK	19
7	REFERENCES	20

ABSTRACT

Organ transplantation requires careful coordination among donor-organ availability, recipient compatibility, hospital resources, and time-sensitive transport. Handling these details across separate records can make it difficult for coordinators to compare candidates, explain recommendations, and plan movement between hospitals. IntelliTransplant is a web-based decision-support prototype designed to organize this workflow through a shared interface and a set of AI-inspired computational engines.

The application provides modules for organ registration, recipient registration, hospital management, organ–recipient matching, and transport planning. The organ module stores organ type, blood group, donor information, hospital association, status, and registration time. Donor identifiers are generated automatically by the backend, while database records remain the source of truth. The recipient module captures relevant matching attributes such as required organ, blood group, urgency, PRA percentage, age, and hospital association. Hospital records include location coordinates and contact details.

The matching interface presents a Compatibility Score produced by the Constraint Satisfaction Problem (CSP) assignment component and displays the selected organ and recipient with an explanation of the compatibility decision. Backtracking helps explore possible assignments while satisfying predefined constraints. The system also includes a route-planning view that identifies a recommended route using A* search and shows distance, estimated travel time, and model-estimated risk/success probabilities. Uniform Cost Search (UCS) provides a cost-based comparison path, while a Bayesian component estimates risk based on predefined factors. A coordination agent combines outputs from these components to support a more organized decision-making workflow.

The application uses React for the frontend, FastAPI for backend services, and a SQL database for storing and retrieving information. Its modular design supports structured data management and integration of different classical AI techniques within a single interface. IntelliTransplant demonstrates how search algorithms, constraint-based matching, and probabilistic reasoning can support transplant coordination.

The prototype is intended for academic demonstration and must not be treated as a clinical allocation authority or as a substitute for transplant-team judgement, validated medical protocols, live traffic services, or regulatory requirements.

CHAPTER 1
INTRODUCTION

1.1 Background

Organ transplantation is a coordinated process involving donor identification, recipient evaluation, compatibility assessment, hospital communication, and transport of the organ within a limited time window. Information about these activities may be recorded in different systems or manually exchanged among people. A unified coordination interface can make important records easier to review and reduce the effort required to move between separate workflows. IntelliTransplant brings these activities together through a common interface supported by classical AI techniques.

1.2 Problem Statement

The project addresses the challenge of presenting donor-organ data, recipient needs, hospital details, matching results, and transport recommendations in one understandable application. The system should avoid requiring users to manually invent identifiers when the backend can generate them, display the records returned by the database, and show why a match or route has been recommended. It also aims to simplify the comparison of matching candidates and transport options through structured information and algorithm-based results.

1.3 Objectives

- Create a web interface for registering and viewing available organs.
- Maintain recipient records and relevant matching attributes.
- Maintain hospital records, including location and contact details.
- Use constraint-based logic to score or select compatible organ–recipient pairs.
- Apply graph-search methods to recommend transport routes between hospitals.
- Present risk estimates and decision explanations in a readable format.
- Connect the frontend to a backend API and persistent database.
- Integrate the outputs of different AI components to support coordinated decision-making.

1.4 Scope

The current scope is a prototype for academic demonstration. It focuses on data entry, database-backed lists, algorithmic recommendations, and explainable output screens. It demonstrates the application of constraint satisfaction, search algorithms, and probabilistic reasoning to a healthcare coordination problem. Real-world clinical deployment would require verified medical rules, secure access control, auditability, formal validation, and integration with authorized transplant systems. The prototype is not intended to replace clinical judgement or official organ-allocation procedures.

CHAPTER 2
LITERATURE REVIEW

2.1 Constraint Satisfaction Problems

A Constraint Satisfaction Problem represents variables, domains, and constraints. In a matching context, candidate pairs can be filtered by hard constraints such as required organ type and blood-group compatibility. Additional criteria can be used to rank feasible candidates. CSP methods are useful because they make eligibility conditions explicit instead of hiding them inside an opaque score.

2.2 Heuristic Search and Path Planning

A* search evaluates paths using a known cost so far and a heuristic estimate of the remaining cost. It is commonly used in route finding when the heuristic is suitable for the problem. Uniform Cost Search expands the lowest accumulated-cost path and is useful when edge costs represent distance, time, or another non-negative transport cost. Both are graph-search methods, but they do not have to return the same route when their cost functions or search guidance differ.

2.3 Bayesian Reasoning

Bayesian reasoning updates the probability of a hypothesis using observed evidence and prior information. In a prototype, a Bayesian component may present an estimated transport-risk value based on configured input factors. Such values are meaningful only to the extent that the model, variables, and parameters are defined and validated; a displayed percentage should not be interpreted as a medically validated probability without evidence.

2.4 Web Application Technologies

React supports reusable user-interface components and state-driven rendering. FastAPI provides Python-based HTTP endpoints and request/response validation. SQLAlchemy can map Python classes to relational database tables and manage database operations. Together, these technologies allow a frontend to submit structured forms to a backend API and render the persisted records returned by that API.

2.5 Project Positioning

IntelliTransplant brings these techniques into a single demonstrator: CSP-based matching, search-based transport planning, Bayesian-style risk analysis, and a coordination layer. The interface emphasizes visible scores and explanations so that a user can inspect the basis of a recommendation.

CHAPTER 3
PROPOSED SYSTEM

3.1 Overview

The proposed system is a client–server web application. The React frontend presents the dashboard and the Organ, Recipient, Hospital, Matching, and Transport pages. It calls backend endpoints through a centralized API service. The FastAPI backend validates incoming data using schemas, executes application logic, and reads or writes records through SQLAlchemy models and a relational database.

3.2 Main Functional Flow

1. Register hospitals so their IDs and location data can be associated with organs and recipients.
2. Register organs with organ type, blood group, donor name/contact, and hospital association; donor ID is generated by backend logic.
3. Register recipients with required organ, blood group, urgency, PRA percentage, and hospital information.
4. Request a matching operation; apply compatibility constraints and calculate a score for eligible candidates.
5. Request transport planning for the selected origin and destination hospitals.
6. Display route, estimated distance/time, risk estimate, and decision explanation on the frontend.

3.3 Design Principles

The design prioritizes traceability, clear forms, reusable components, separate page-level CSS, and backend-owned data. Frontend fields should correspond to the API schema rather than being invented independently. Lists should be populated from GET responses, and successful POST operations should refresh the displayed records.

3.4 Important Safety Boundary

The software is a student prototype. It demonstrates computational methods and interface flow; it does not establish clinical eligibility, organ-allocation policy, or real transport readiness. All clinical decisions must remain with authorized professionals using approved protocols.

3.1 SYSTEM ARCHITECTURE AND WORKFLOW

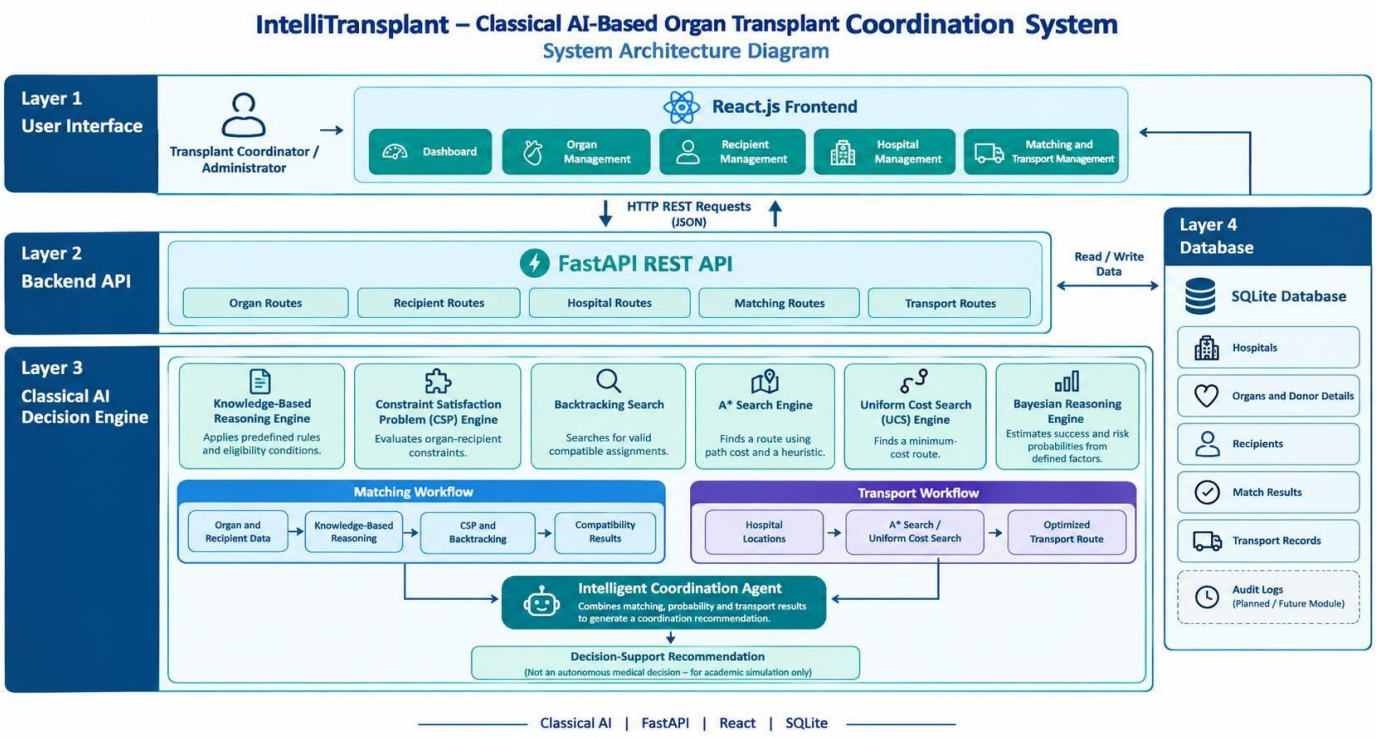


Figure 3.1: Logical architecture of IntelliTransplant (conceptual representation based on the implemented screens and API snippets shared for the project).

3.1.1 Data Flow

A user action begins in a React page. The page calls a service function that sends an HTTP request to a FastAPI route. Pydantic schemas validate the request fields. The route calls application logic and uses the database session to query or commit records. The backend serializes the response to JSON, and the frontend updates its state to display the latest result.

3.1.2 Organ Registration Example

The Organ form sends organ_type, blood_group, donor_name, donor_contact, and hospital_id. The backend generates donor_id, persists the Organ row, and returns fields such as id, donor_id, status, and created_at. The registered-organs table should render the returned response rather than creating fake rows locally.

3.1.3 AI Output Flow

Matching and transport outputs are displayed as decision-support information. The frontend should label estimates clearly, preserve the actual values returned by the backend, and avoid presenting placeholder or hard-coded values as live calculations.

CHAPTER 4
MODULES DESCRIPTION

4.1 ORGAN MANAGEMENT MODULE

The Organ Management page supports registration and review of donor organs. Its form includes organ type, blood group, donor name, donor contact, and hospital ID. The registered-organs table shows database-backed entries with organ ID, type, blood group, donor ID, donor name, contact, hospital association, and status.

The backend model shared for the project defines an organs table with an integer primary key, organ_type, blood_group, donor_id, optional donor_name, optional donor_contact, hospital_id, status, and created_at. The OrganResponse schema exposes the saved record, including its generated ID and creation timestamp.

4.1.1 Automatic Donor Identifier

The user requirement is that donor IDs are generated automatically. Therefore, donor_id should not be a required field in the frontend form. It should be generated by backend logic during creation and returned in the POST response. The frontend can then display it in the registered-organs table. The exact identifier format should be implemented consistently in the backend and protected against duplicate generation.

4.1.2 Validation and Error Handling

- Require organ type, blood group, and hospital association.
- Validate hospital_id as a valid numeric ID associated with an existing hospital, if that relationship is enforced.
- Show a clear success or failure message based on the API response.
- Reload the organ list after a successful creation so the database state is visible.
- Do not silently substitute sample data when the API fails.

4.1.3 Result

The screenshot demonstrates a two-column registration form and a registered-organs table. Sample visible rows include kidney, pancreas, and liver entries, with blood groups, donor identifiers, hospital IDs, and available status.

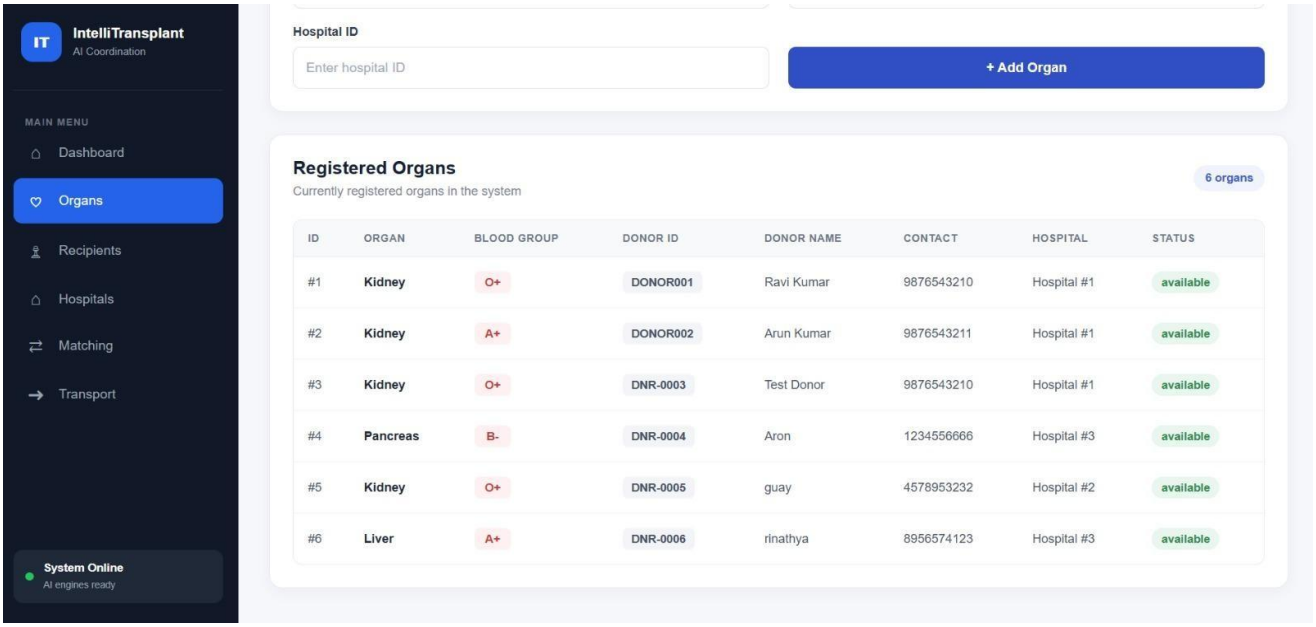


Figure 4.1: Organ registration form and registered-organ list.

4.2 RECIPIENT MANAGEMENT MODULE

The Recipient Management page records the needs of people awaiting transplantation. The visible form includes full name, age, required organ, blood group, urgency level, PRA percentage, and hospital selection. These fields provide structured inputs for later compatibility checks and priority calculations.

4.2.1 Relevant Attributes

- Required organ: used to reject organ types that do not satisfy the recipient's need.
- Blood group: used by compatibility rules configured in the matching logic.
- Urgency level (1–5): represents a priority-related input; its clinical meaning must be defined by the project rules.
- PRA percentage: a sensitization-related value displayed in the matching result; the prototype's exact formula must be documented from the backend implementation.
- Hospital association: identifies the recipient's registered hospital for coordination and route planning.

4.2.2 Identifier and Data Integrity

Recipient IDs should be generated consistently by the backend or database rather than manually invented by the user. The frontend should submit only fields required by the recipient creation schema. It should refresh the recipient list after saving and use the returned ID for future matching requests.

4.2.3 User Interface Result

The screen organizes the form into paired fields, with a summary area for total recipients, waiting-list count, and high-urgency recipients. These summary values should be derived from API data rather than hard-coded values when connected to the live backend.

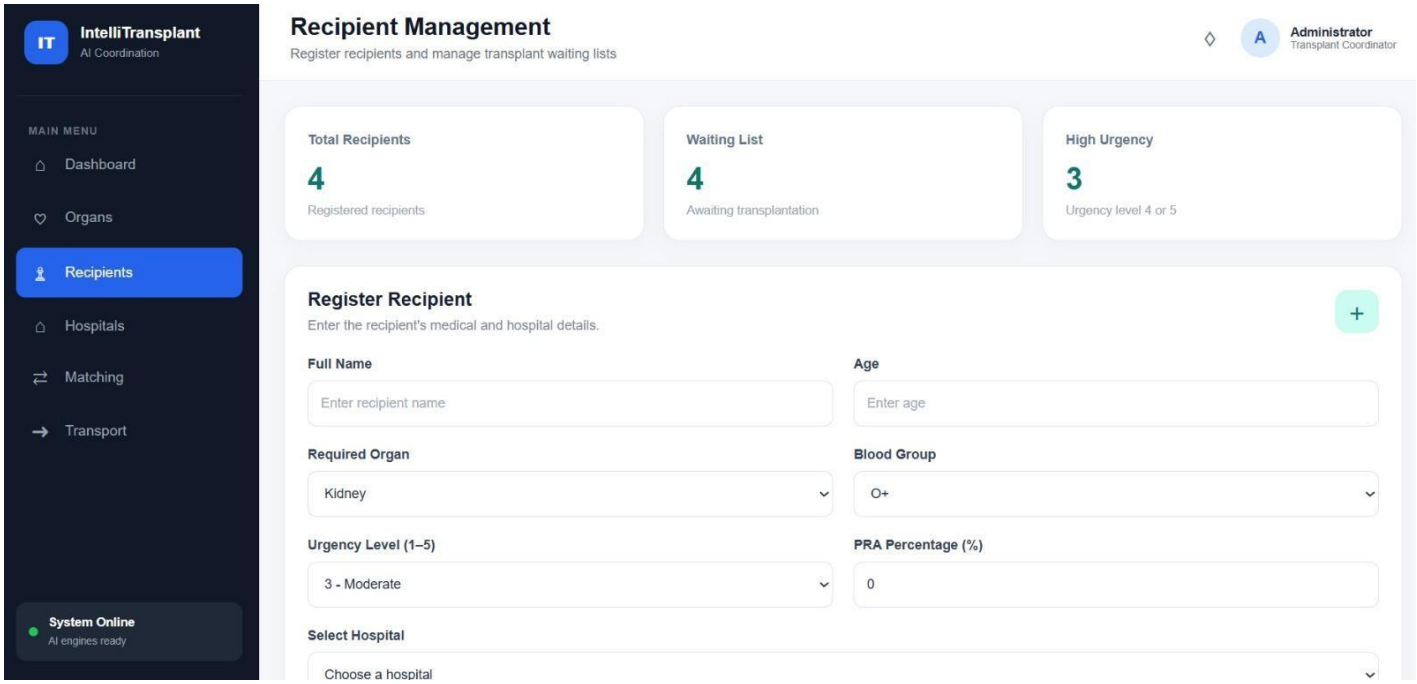


Figure 4.2: Recipient registration and waiting-list interface.

4.3 HOSPITAL MANAGEMENT MODULE

Hospital Management maintains the hospitals participating in the coordination workflow. The visible form includes hospital name, city, latitude, longitude, and optional contact number. The registered-hospitals table presents a hospital identifier, name, city, coordinates, and contact information.

4.3.1 Why Hospital Coordinates Matter

Latitude and longitude can be used to represent hospital locations as graph nodes or geographic points for route planning. A route engine can use distance between locations as an edge cost or heuristic, provided that the graph and cost calculation are implemented. Coordinates alone do not provide live road distance, traffic, ambulance availability, or real-world transport readiness.

4.3.2 Hospital IDs

The UI shows readable identifiers such as HSP-0001 while the organ schema currently uses an integer hospital_id. The application must keep this distinction explicit: a displayed code may map to an internal numeric primary key, but API requests must send the type expected by the backend schema. The frontend should not assume that a formatted code can be passed where an integer is required.

4.3.3 Result

The screenshot shows a registration form and four example hospital records in Chennai, with coordinates and contact numbers. These are demonstration records in the screenshot; they should not be interpreted as verified operational transplant locations.

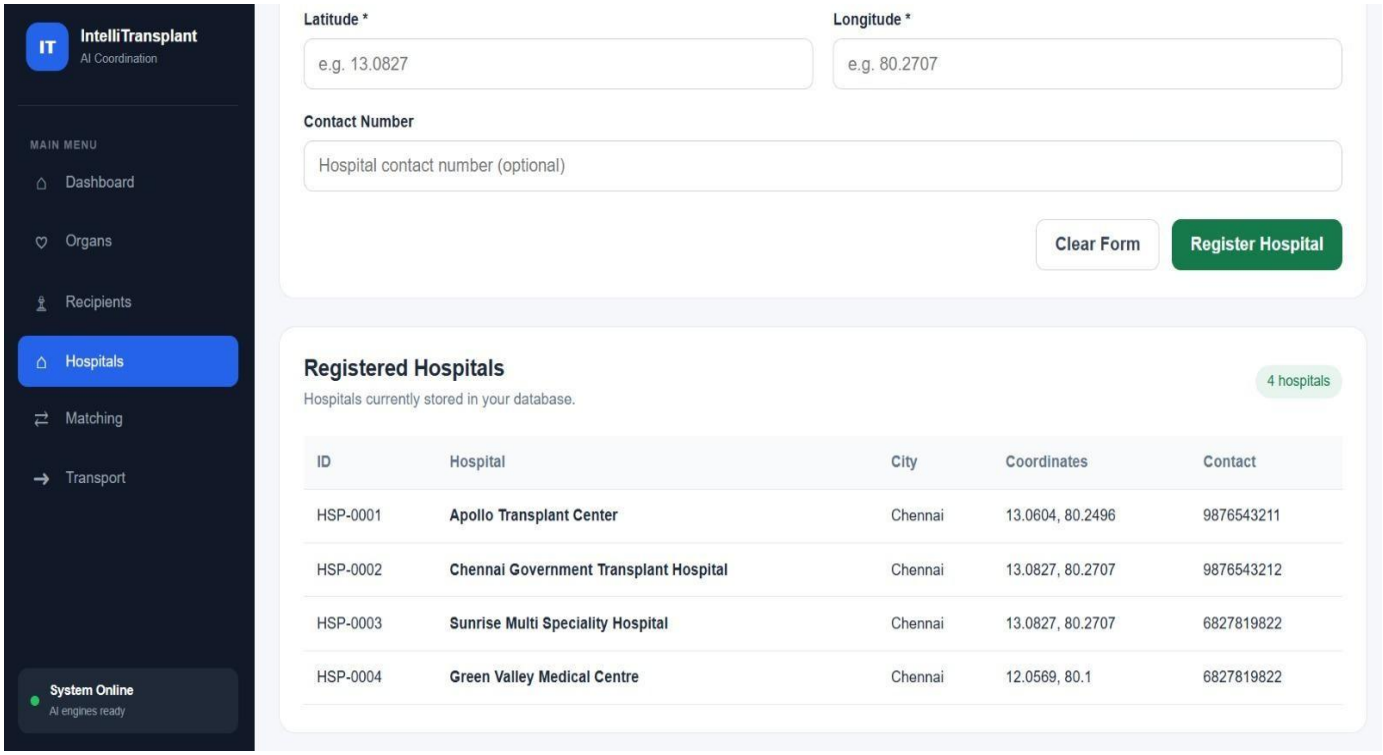


Figure 4.3: Hospital registration and database-backed hospital table.

4.4 AI MATCHING AND CSP SCORE

The Matching page requests a match from the backend and displays a Compatibility Score, selected organ ID, recipient ID, blood-group compatibility, PRA percentage, recipient details, and a decision explanation. The example screenshot shows a score of 99 and an explanation that includes organ-type compatibility, blood-group compatibility, urgency level, and PRA percentage.

4.4.1 Role of the CSP Engine

A Constraint Satisfaction Problem engine represents matching requirements as constraints. In this project, the engine can first identify feasible organ–recipient pairs using hard rules such as required organ type and configured blood-group compatibility. It can then apply the project's scoring or ranking logic to compare eligible candidates. A score is only interpretable when the backend documents its components, weights, ranges, and maximum possible value.

4.4.2 Score Interpretation

The frontend screenshot labels the displayed value as the score returned by CSP assignment. The interface should therefore show the backend value and avoid assuming a denominator such as 100 or 200 unless the implementation defines it. If the score is intended to be out of 100, the backend and UI should both document and enforce that range. The PRA and urgency contributions must likewise be described from the actual scoring function.

4.4.3 Explainability

The decision explanation is important because users should be able to see why a candidate was selected. Explanations should be generated from the actual checks and values used by the backend,

not from a generic sentence that may disagree with the result. A matching score is a software output, not a clinical recommendation for actual transplantation.

.

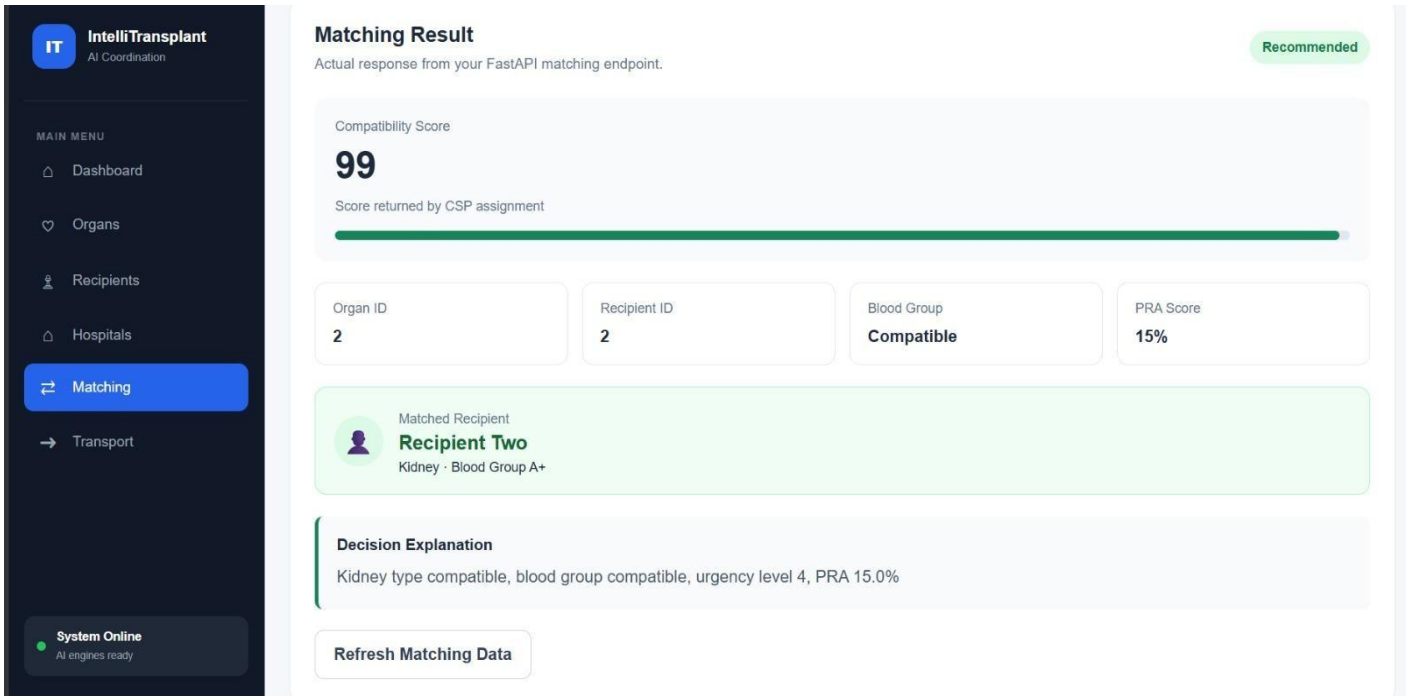


Figure 4.4: Matching result with CSP compatibility score and decision explanation

4.5 TRANSPORT PLANNING AND RISK ANALYSIS

The Transport page presents an optimized transport plan between an origin hospital and a destination hospital. The visible result includes a transport identifier, route status, distance, estimated travel time, risk probability, success probability, and a recommended route with departure and destination nodes.

4.5.1 A* Search

A* is a best-first graph-search algorithm that evaluates a candidate path using $f(n) = g(n) + h(n)$, where $g(n)$ is the known path cost from the start and $h(n)$ estimates the remaining cost to the goal. For hospital routing, the cost might represent distance or travel time, while a geographic heuristic can estimate remaining distance. The chosen heuristic must be compatible with the edge-cost definition.

4.5.2 Uniform Cost Search (UCS)

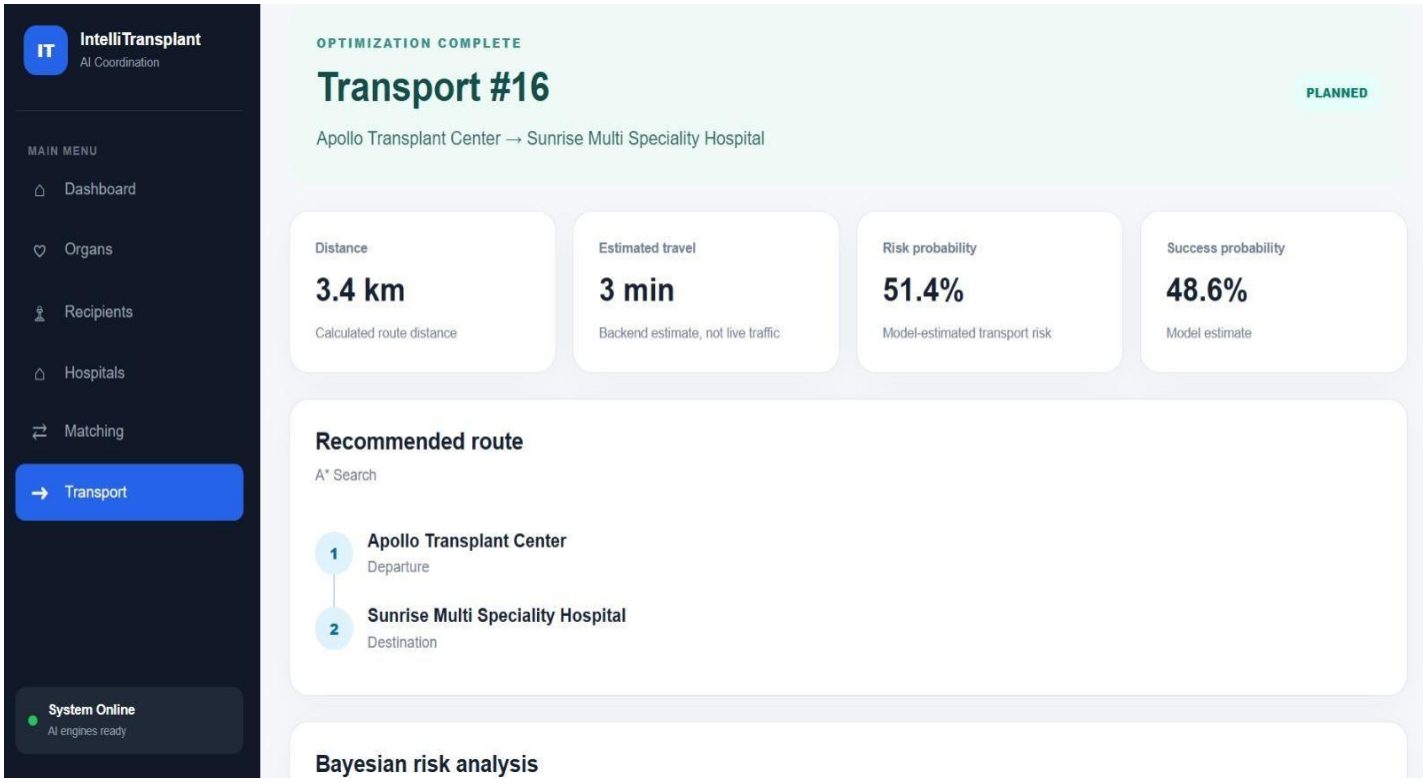
UCS expands the frontier node with the smallest accumulated path cost $g(n)$. It is useful when the objective is to minimize a non-negative route cost. A* and UCS are related but not redundant in every configuration: UCS uses accumulated cost only, while A* also uses a heuristic. If both are run, their outputs can be compared for path cost or used for evaluation; their results should not be combined blindly. One algorithm may be sufficient if the system has a single clearly defined route objective.

4.5.3 Bayesian Risk Estimate

The transport screen displays model-estimated risk and success probabilities. A Bayesian engine can combine prior assumptions and observed factors to estimate a probability, but the screenshot alone

does not reveal the model's variables, priors, conditional probabilities, or training/validation procedure. Therefore, the report describes these values as prototype estimates and does not claim clinical validation.

Figure 4.5: Transport route, estimated distance/time, and model-estimated risk.



4.6 COORDINATION AGENT AND AI ENGINE INTEGRATION

The dashboard lists five configured components: Knowledge Engine, CSP Engine, A*/UCS Engine, Bayesian Engine, and Coordination Agent. Their roles in the proposed workflow are described below. 'Configured' in the interface indicates that the components are represented in the application; it does not by itself prove that every component has been independently validated or is actively running.

4.6.1 Knowledge Engine

Stores or applies domain rules used by the workflow, such as organ categories, blood-group compatibility rules, and field definitions. The knowledge base should keep medical or allocation rules traceable to authorized sources and should distinguish hard eligibility constraints from soft ranking preferences.

4.6.2 CSP Engine

Checks candidate organ–recipient pairs against constraints and returns feasible candidates or an assignment score. Its implementation should state the constraints, score formula, weights, tie-breaking behaviour, and score range.

4.6.3 A* / UCS Engine

Finds a route between hospital nodes according to a cost function. A* uses a heuristic in addition to path cost; UCS uses path cost alone. Route results depend on the graph, edge weights, and heuristic supplied.

4.6.4 Bayesian Engine

Produces a probability-style risk estimate from defined evidence and model parameters. Its outputs should be labelled estimates, with assumptions documented and validated before any high-stakes use.

4.6.5 Coordination Agent

Acts as the workflow orchestrator: obtains or references the relevant records, invokes matching and transport components in the appropriate sequence, and assembles a result for the UI. The agent should preserve the origin of each output and handle missing or failed engine responses clearly.

4.6.6 Integration Principle

The components have distinct jobs rather than five engines independently calculating the same score. The CSP component addresses matching constraints, search algorithms address route costs, Bayesian reasoning addresses uncertainty, the knowledge engine supplies rules, and the coordination agent connects the workflow.

CHAPTER 5
IMPLEMENTATION AND RESULTS

5.1 EXPERIMENTAL SETUP

5.1.1 Technologies

Technology	Purpose
React	Frontend pages and reusable UI components
Vite	Frontend development server and build tooling
CSS	Page/component-specific styling and layout
FastAPI	HTTP API routes and request/response handling
Pydantic	Validation of request and response schemas
SQLAlchemy	ORM access to database models and sessions
Relational database	Persistent storage of application records
Graph search / probabilistic logic	Route planning and risk-estimate components

5.1.2 API and Persistence

The Organ API shown during development exposes GET /organs/ to retrieve stored organ records and POST /organs/ to create a record. The route uses a database session, adds the new Organ model, commits the transaction, refreshes the record, and returns an OrganResponse. The create schema should match the agreed identifier-generation design: donor_id is generated server-side and therefore should not be required from the frontend.

5.1.3 Test Approach

- Open each page and confirm the sidebar remains visible and navigation works.
- Load organ, recipient, and hospital lists from the backend and verify displayed rows match database responses.
- Submit valid and invalid form data and check the resulting API status and error message.
- Create a record and refresh the page to confirm it persists.
- Run matching and route planning with known demonstration records and inspect returned explanations.
- Verify that failure states are shown instead of misleading placeholder values.

5.2 DASHBOARD AND ORGAN REGISTRATION RESULTS

5.2.1 Dashboard

The dashboard provides an overview of the coordination system. It displays summary cards for available organs, waiting recipients, registered hospitals, and active transports. A recent-matches panel provides a path into the matching workflow, while the AI Engines panel lists the five configured components. Counts should be connected to backend data so that they reflect the current database.

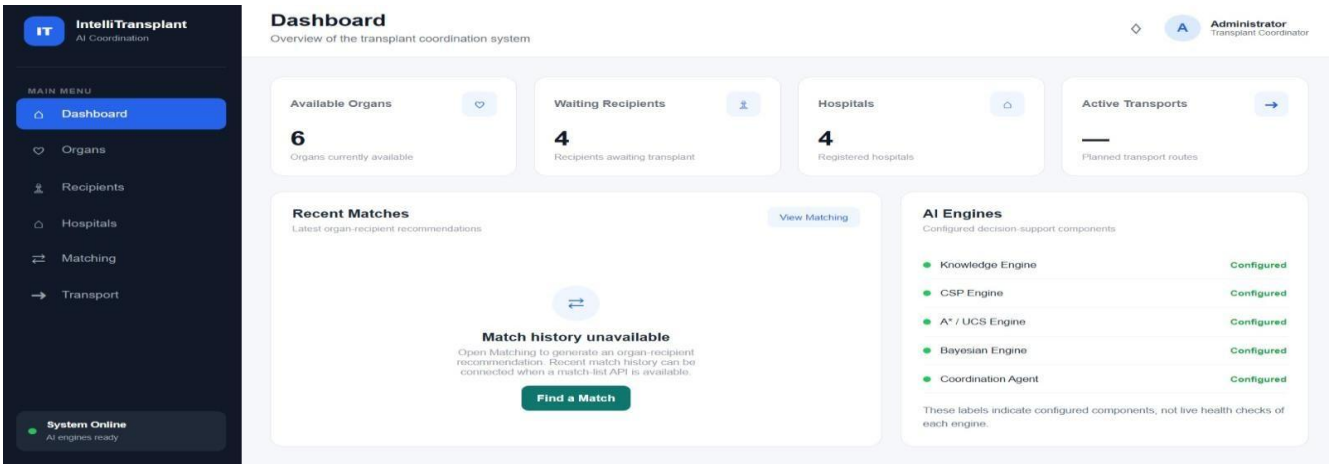


Figure 5.1: IntelliTransplant dashboard with system summary and AI engine list.

5.2.2 Organ Registration

The organ screen separates data entry from the registered-organ table. This makes it possible to add a new organ while continuing to inspect existing records. The table displays automatically assigned organ IDs and donor IDs, as well as donor name/contact and hospital association. This is consistent with the requirement that IDs be generated by the backend and displayed after successful creation.

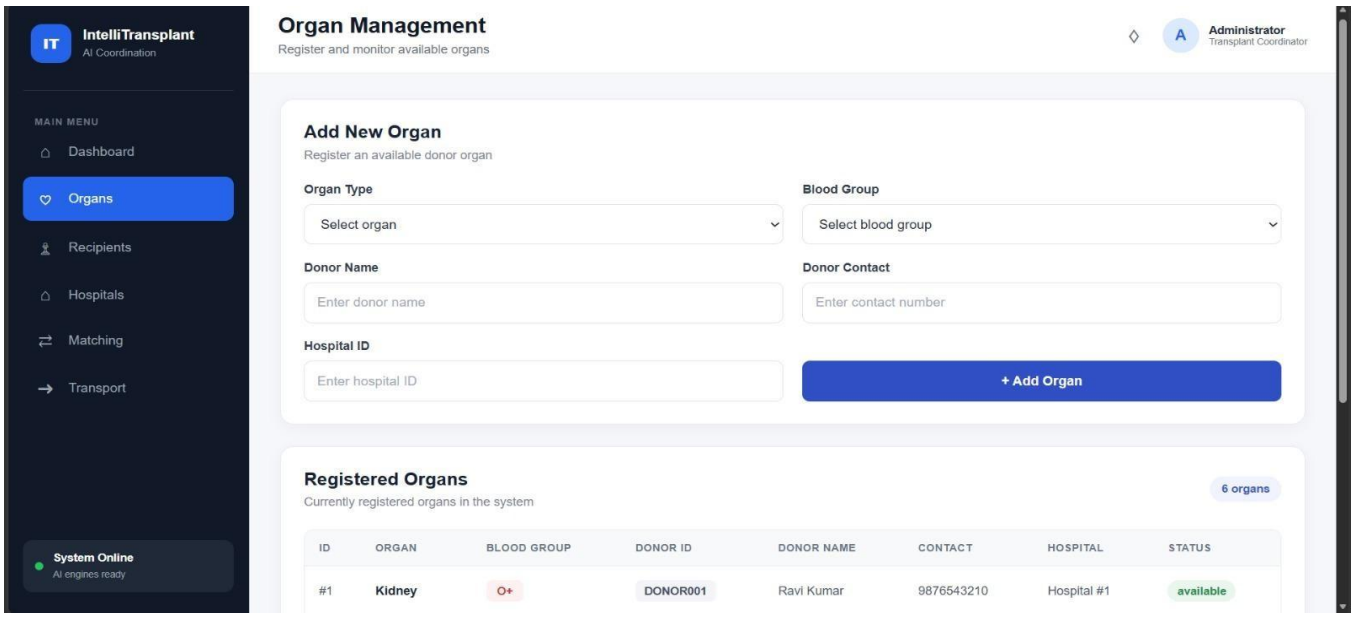


Figure 5.2: Registered-organ records displayed in the application.

5.2.3 Observed Result

The supplied screenshot shows six registered organ records and a count badge. The screenshot is evidence of the UI state at capture time; the live database may change as records are added or edited.

5.3 RECIPIENT AND HOSPITAL RESULTS

5.3.1 Recipient Management

The recipient screen presents summary cards and a structured form for entering recipient information. The urgency and PRA fields are especially relevant to the project's matching logic. Their contribution to any score must come from the backend formula and must not be inferred only from the visual interface.

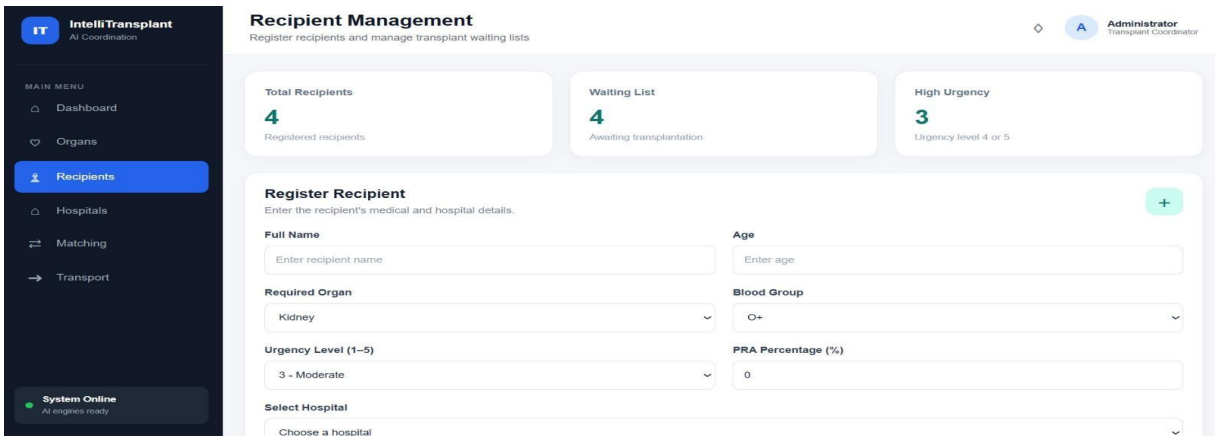


Figure 5.3: Recipient registration form and waiting-list summary.

5.3.2 Hospital Management

The hospital screen supports registration and review of hospital names, cities, coordinates, and contact details. These records provide the locations used by the transport workflow and the hospital associations referenced by organ and recipient records.

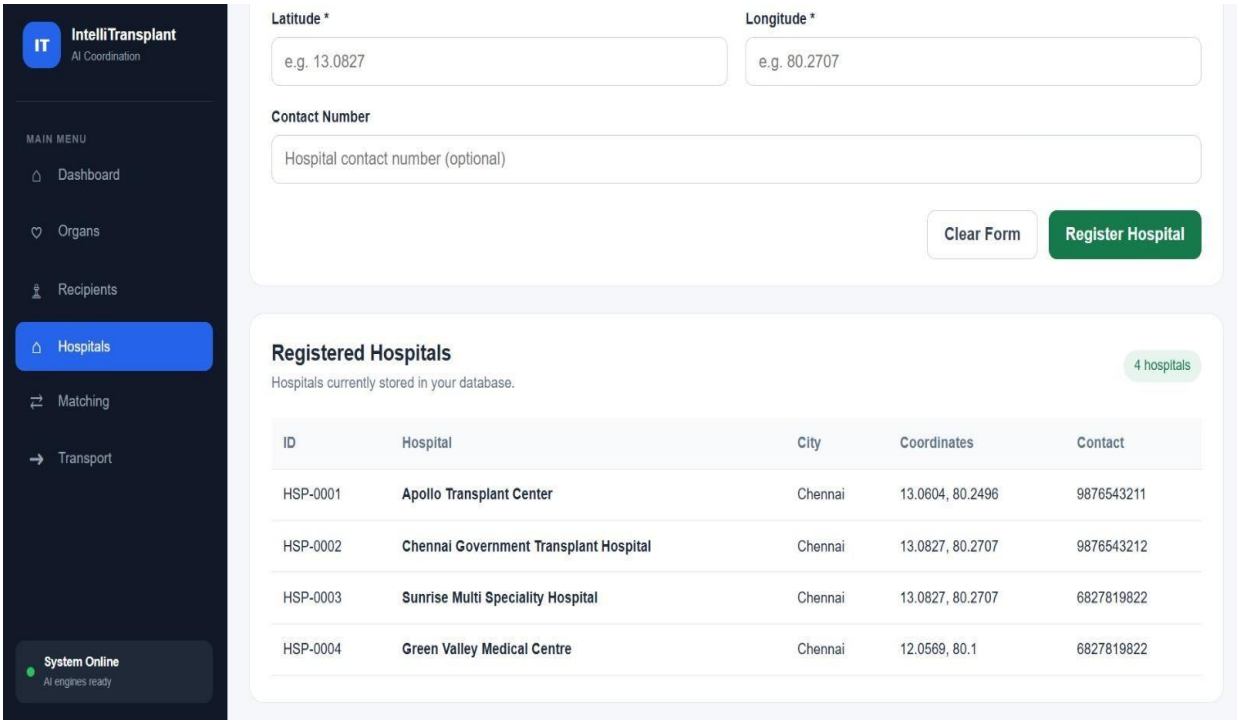


Figure 5.4: Registered hospital list and location information.

5.3.3 Observed Result

The provided screenshots show four recipients in the summary and four hospitals in the registered-hospital list. These counts describe the captured demonstration environment, not a guaranteed count in future runs.

5.4 MATCHING AND TRANSPORT RESULTS

5.4.1 Matching Result

The matching screenshot shows a recommended candidate with compatibility score 99, organ ID 2, recipient ID 2, a compatible blood-group label, and PRA value 15%. The decision explanation lists organ type, blood-group compatibility, urgency level 4, and PRA 15.0%. This output illustrates how the interface makes the matching decision inspectable. The maximum possible score and exact scoring formula should be confirmed directly from the backend implementation before describing the score as a percentage.

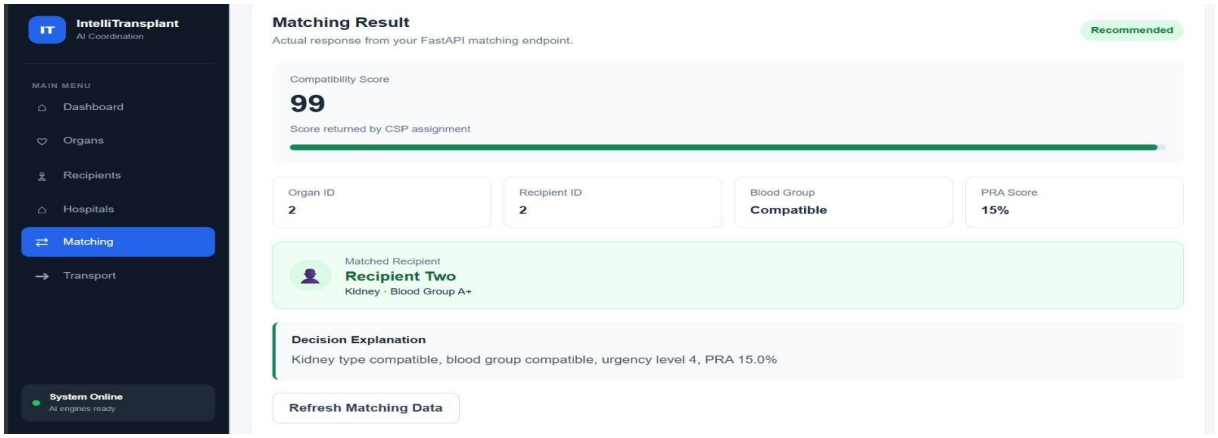


Figure 5.5: Example matching result returned by the backend.

5.4.2 Transport Result

The transport screenshot shows Transport #16 with planned status, a route between Apollo Transplant Center and Sunrise Multi Speciality Hospital, distance 3.4 km, estimated travel time 3 minutes, risk probability 51.4%, and success probability 48.6%. It labels the route method as A* Search and identifies departure and destination nodes.

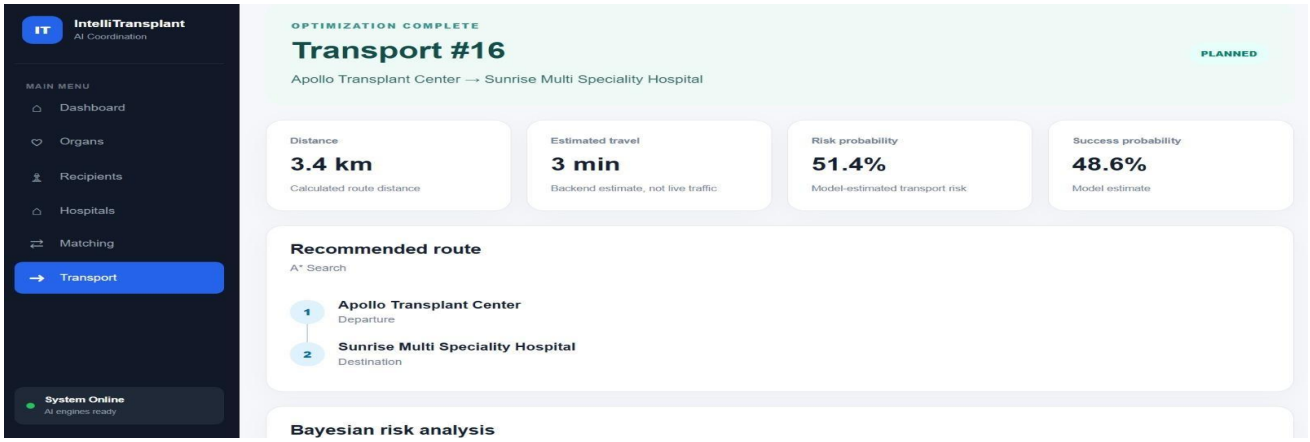


Figure 5.6: Example route-planning and risk-analysis result.

5.4.3 Result Interpretation

The displayed travel time is identified as a backend estimate rather than live traffic. Risk and success are model estimates. They should not be interpreted as validated operational probabilities without model documentation and testing. The interface demonstrates end-to-end presentation of route and matching outputs, while further testing is needed to verify edge cases, algorithmic optimality, and data integrity.

CHAPTER 6
CONCLUSION AND FUTURE WORK

6.1 Conclusion

IntelliTransplant demonstrates a unified web interface for organ records, recipient requirements, hospital information, matching results, and transport planning. The application combines a React frontend with a FastAPI backend and database-backed records. Its conceptual AI pipeline assigns distinct responsibilities to domain knowledge, CSP-based matching, graph search, Bayesian-style risk estimation, and workflow coordination.

The supplied screenshots show the intended user experience: a dashboard, forms and registered-record tables, a compatibility result with a decision explanation, and a transport plan with route and model-estimated values. Backend-generated identifiers and database-backed refreshes are important to ensuring that the interface reflects persisted records rather than disconnected sample data.

6.2 Limitations

- The exact CSP scoring formula and maximum score must be confirmed from source code.
- The Bayesian model's variables, priors, and validation evidence are not visible in the screenshots.
- Route estimates do not represent live traffic or guaranteed transport time.
- Demonstration hospital and donor records must not be treated as verified real-world clinical records.
- A production system would require authentication, authorization, audit logs, privacy safeguards, and formal clinical/regulatory review.

6.3 Future Work

- Document and unit-test the CSP score formula, score bounds, compatibility rules, and tie-breaking logic.
- Add comprehensive backend validation and explicit relationships between organs, recipients, and hospitals.
- Add secure authentication, role-based access, audit trails, and privacy-aware handling of personal information.
- Evaluate A* and UCS against known test graphs and document route cost units and heuristic assumptions.
- Calibrate and validate risk estimates with appropriate authorized data before any real-world use.
- Add automated tests, API error handling, responsive layouts, and clearer loading/empty/error states.

CHAPTER 7 – REFERENCES

1. Dechter, R. (2003). *Constraint Processing*. Morgan Kaufmann.
2. Nilsson, N. J. (1998). *Artificial Intelligence: A New Synthesis*. Morgan Kaufmann.
3. Pearl, J. (1988). *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference*. Morgan Kaufmann.
4. Poole, D. L., & Mackworth, A. K. (2017). *Artificial Intelligence: Foundations of Computational Agents* (2nd ed.). Cambridge University Press. https://artint.info/2e/html/ArtInt2e.html
5. United Network for Organ Sharing. (n.d.). *How we match organs*. https://unos.org/transplant/how-we-match-organs/
6. Python Software Foundation. (n.d.). *Python documentation*. https://docs.python.org/3/
7. SQLite. (n.d.). *SQLite documentation*. https://www.sqlite.org/docs.html
8. FastAPI. (n.d.). *SQL (Relational) Databases*. https://fastapi.tiangolo.com/tutorial/sql-databases/
9. Mozilla. (n.d.). *HTTP: Overview*. MDN Web Docs. https://developer.mozilla.org/en-US/docs/Web/HTTP/Overview
10. World Health Organization. (2024). *WHO guiding principles on human cell, tissue and organ transplantation*. https://www.who.int/health-topics/transplantation